

TP en Clase: Coding Agent

Construir un agente de IA desde cero, sin frameworks

Objetivo

Implementar un **harness** (loop de orquestación) que conecte un LLM con herramientas para resolver tareas de código de forma autónoma. El agente debe ser genérico: recibe una instrucción en lenguaje natural y usa tools para cumplirla.

No se permite el uso de frameworks como LangChain, CrewAI, ni similares. Se puede usar cualquier lenguaje (Python, Node.js, etc.) y cualquier API de LLM.

¿Qué es un harness?

Un harness es el código que orquesta la interacción entre tu programa y un LLM. Es un loop que envía mensajes al modelo, detecta cuándo el modelo quiere usar una herramienta, la ejecuta, le devuelve el resultado, y repite hasta que la tarea esté completa.

El flujo básico es:

1. Enviar mensajes al LLM junto con la definición de tools disponibles.
2. Si el LLM responde pidiendo usar una tool, ejecutarla localmente.
3. Devolver el resultado de la tool al LLM.
4. Repetir hasta que el LLM responda con un mensaje final (sin pedir tools).

Modo conversación

El agente debe funcionar como un **chat interactivo**. Cuando el agente termina una tarea (el LLM responde sin pedir tools), el programa no debe terminar: debe mostrar la respuesta y esperar un nuevo mensaje del usuario.

El historial de mensajes se mantiene entre turnos. Esto permite repreguntar, pedir correcciones, dar instrucciones de seguimiento, o cambiar de tarea sin perder el contexto de lo que el agente ya hizo.

Es decir: hay dos loops anidados. El loop interno ejecuta tools hasta que el LLM termina su turno. El loop externo espera input del usuario y vuelve a arrancar el loop interno con el mensaje nuevo.

Tools a implementar

El agente debe contar con las siguientes herramientas. La implementación es libre (pueden usar el lenguaje y la API que prefieran):

Tool	Descripción
read_file	Lee el contenido de un archivo dado su path.

write_file	Escribe contenido en un archivo, reemplazando su contenido actual.
run_command	Ejecuta un comando de terminal y devuelve el output (stdout y stderr).
list_files	Lista los archivos en un directorio.
web_search	Busca información en la web y devuelve los resultados. Se sugiere usar la API de Tavily (tavily.com).

Plan mode

El agente debe soportar un **plan mode** que se pueda activar o desactivar. Cuando está activado, antes de ejecutar cualquier acción el agente arma un plan de pasos y se lo muestra al usuario. El usuario puede aprobarlo, modificarlo, o rechazarlo. Recién después de la aprobación el agente empieza a ejecutar tools. Cuando está desactivado, el agente ejecuta directamente.

Supervisión (human in the loop)

El agente debe soportar un modo de **supervisión** que se pueda activar o desactivar. Cuando está activado, el agente pide confirmación al usuario antes de ejecutar acciones que modifiquen el sistema: escribir archivos (write_file), ejecutar comandos (run_command). Las acciones de solo lectura (read_file, list_files, web_search) se ejecutan sin pedir permiso. Cuando está desactivado, todas las acciones se ejecutan directamente.

¿Qué probar con el agente?

Una vez que el harness funcione, pruébenlo con distintas tareas para ver cómo se comporta el loop. Algunas ideas:

Tarea	Por qué es interesante
Corregir un bug	El agente tiene que leer el código, correr los tests, entender qué falla, editar, y volver a correr.
Escribir código nuevo	Pedirle que implemente una función o módulo desde cero y que verifique que ande corriendo tests.
Leer documentación y configurar un proyecto	Darle un proyecto con un README y que el agente lea las instrucciones, instale dependencias (npm install, pip install, etc.) y lo haga correr.
Refactorizar código	Que lea un archivo, lo mejore, y verifique que los tests sigan pasando después de los cambios.
Explorar un repo desconocido	Darle un proyecto que no conoce y pedirle que explique qué hace. Tiene que navegar archivos, leer código y armar una respuesta.
Agregar tests	Darle un archivo sin tests y pedirle que escriba tests, los corra, y arregle lo que falle.

Buscar e implementar	Pedirle que busque en la web cómo usar una librería o API que no conoce, y que implemente algo con eso.
----------------------	---

Extra opcional

Feature	Descripción
Guardrails vía archivo de configuración	Agregar un archivo de configuración (JSON, YAML, o similar) que defina restricciones para el agente. Por ejemplo: • Archivos o directorios a los que no tiene acceso (read/write bloqueado) • Comandos prohibidos (rm -rf, git push, etc.) • Directorios permitidos (el agente solo puede operar dentro de una carpeta específica) El harness debe leer esta config al arrancar y validar cada tool call antes de ejecutarla.

Entregables

1. El código del harness completo y funcionando.
2. Al menos dos corridas distintas del agente con tareas diferentes (copiar el output de la terminal).
3. Responder brevemente: ¿Cuántas iteraciones del loop hizo el agente en cada caso? ¿Qué salió bien y qué salió mal? ¿Qué mejorarían?

Duración estimada: 3 horas — Modalidad: individual o en parejas